Capítulo 2:

Introdução aos aplicativos Java

OBJETIVOS

Neste capítulo você aprenderá:

- Como escrever aplicativos Java simples.
- Como utilizar instruções de entrada e saída.
- Tipos primitivos do Java.
- Conceitos básicos de memória.
- Como utilizar operadores aritméticos.
- Precedência dos operadores aritméticos.
- Como escrever instruções de tomada de decisão.
- Como utilizar operadores relacionais de igualdade.

Agenda

- 2.1 Introdução
- 2.2 Primeiro programa Java: imprimindo uma linha de texto
- 2.3 Modificando nosso primeiro programa Java
- 2.4 Exibindo texto com `printf`
- 2.5 Outros aplicativos Java: adicionando inteiros
- 2.6 Conceitos de memória
- 2.7 Aritmética
- 2.8 Tomada de decisão: operadores de igualdade e operadores relacionais
- 2.9 (Optional) Software Engineering Case Study: Examining the Requirements Document
- 2.10 Conclusão

2.1 Introdução

- Programação de aplicativo Java
  - Mostrando mensagens
  - Obtendo informação do usuário
  - Cálculos aritméticos
  - Tomadas de decisão

2.2 Primeiro programa Java: Imprimindo uma linha de texto

- Aplicativo
  - Executa quando você usa o comando `java` para carregar a Java Virtual Machine (JVM)
- Programa simples
  - Mostra uma linha de texto
  - Ilustra vários recursos importantes da linguagem Java

Sumário

Welcome1.java

```java
// Fig. 2.1: Welcome1.java
// Programa de impressão de texto.

public class Welcome1
{
   // método principal inicia a execução do aplicativo Java
   public static void main( String args[] )
   {
      System.out.println( "Bem-vindos a programação Java!" );

   } // fim do método principal

} // fim da classe Welcome1
```

```
Bem-vindos a programação Java!
```

2.2 Primeiro programa Java: Imprimindo uma linha de texto (Cont.)

```
1   // Fig. 2.1: Welcome1.java
```

- Comentários começam com: `//`
  - Comentários são ignorados durante a execução do programa
  - Documentam e descrevem o código
  - Aprimorar legibilidade
- Comentários tradicionais: `/* ... */`

```
/* Isso é um comentário
   tradicional. Ele pode ser
   dividido em muitas linhas */
```

```
2   // Programa de impressão de texto.
```

- Outro comentário de linha
- Nota: números de linhas não fazem parte do programa, foram adicionados como referência.

Erro comum de programação 2.1

Esquecer um dos delimitadores de um comentário tradicional no estilo Javadoc causa um erro de sintaxe. A sintaxe de uma linguagem de programação especifica as regras da criação de um programa adequado nessa linguagem.

Um erro de sintaxe ocorre quando o compilador encontra código que viola as regras da ling. Java (isto é, sua sintaxe). Nesse caso, o compilador não produz um arquivo .class. Em vez disso, o compilador emite uma mensagem de erro para ajudar o programador a identificar e corrigir código incorreto.

Erros de sintaxe também são chamados de erros de compilador, erros em tempo de compilação ou erros de compilação, porque o compilador os detecta durante a fase de compilação. Não será possível executar seu programa até você corrigir todos os erros de sintaxe.

Boa prática de programação 2.1

Cada programa deve iniciar com um comentário que explica o seu proprósito, autor, data e hora em que o programa foi modificado pela última vez.

2.2 Primeiro programa Java: Imprimindo uma linha de texto (Cont.)

```
3
```

- Linha em branco
  - Facilita a leitura do programa
  - Linhas em branco, espaços, e tabulações são caracteres em branco
    - Ignorados pelo compilador

```
4   public class Welcome1
```

- Começa uma declaração de classe para a classe `Welcome1`
  - Cada programa Java tem pelo menos uma declaração de classe definida por você - programador
  - Keyword: words reserved for use by Java
    - palavra-chave para um nome de classe: `class`
  - Nomes de classes: Maiúscula para cada palavra
    - `SimplesNomeDeClasse`

Boa prática de programação 2.2

Utilize linhas em branco e caracteres de espaço em branco para aprimorar a legibilidade do programa.

2.2 Primeiro programa Java: Imprimindo uma linha de texto (Cont.)

```
4   public class Welcome1
```

- Identificador Java
  - Séries de caracteres consistindo de letras, dígitos, sublinhados (_) e sinais de cifrão ($)
  - Não podem iniciar com dígitos nem conter espaços
  - Exemplos: `Welcome1`, `$value`, `_value`, `button7`
    - `7button` é inválido
  - Java is case sensitive (capitalization matters)
    - `a1` and `A1` are different

Boa prática de programação 2.3

Por convenção, sempre inicie o identificador do nome de uma classe com uma letra maiúscula e inicie cada palavra subseqüente no identificador com uma letra maiúscula. Programadores Java sabem que esses identificadores normalmente representam classes Java, portanto nomear suas classes dessa maneira torna seus programas mais legíveis.

Erro comum de programação 2.2

Java diferencia letras maiúsculas de minúsculas. A não utilização de letras maiúsculas e minúsculas adequadas para um identificador normalmente causa um erro de compilação.

2.2 Primeiro programa Java: Imprimindo uma linha de texto (Cont.)

```
4   public class Welcome1
```

- Salvando arquivos
  - Arquivo deve ter o mesmo nome da classe com a extensão `.java`
  - `Welcome1.java`

```
5   {
```

- Chave esquerda{
  - Começa o corpo de toda classe
  - Chave direita deve terminar cada declaração (linha 13)

Erro comum de programação 2.3

É um erro uma classe *public* ter um nome de arquivo que não é idêntico ao nome da classe (mais a extensão .java) em termos de ortografia e de uso de letras maiúsculas e minúsculas.

Erro comum de programação 2.4

É um erro não terminar um nome de arquivo com a extensão .java para um arquivo que contém uma declaração de classe. Se essa extensão não estiver presente, o compilador Java não será capaz de compilar a declaração de classe.

Boa prática de programação 2.4

Sempre que você digita uma chave de abertura, ou chave esquerda, {, em seu programa, imediatamente digite a chave de fechamento, ou chave direita, }, então reposicione o cursor entre as chaves e dê um recuo para começar a digitação do corpo. Essa prática ajuda a evitar erros devidos à ausência das chaves.

Boa prática de programação 2.5

Recue o corpo inteiro de cada declaração da classe por um nível de recuo entre a chave esquerda e a chave direita o que delimita o corpo da classe. Esse formato enfatiza a estrutura da declaração de classe e torna mais fácil sua leitura.

Boa prática de programação 2.6

Configure uma convenção para o tamanho do recuo que você prefere e então aplique uniformemente essa convenção. A tecla *Tab* pode ser utilizada para criar recuos, mas as paradas de tabulação variam entre editores de textos. Recomendam-se a utilização de três espaços para formar um nível de recuo.

Erro comum de programação 2.5

É um erro de sintaxe se as chaves não ocorrerem em pares correspondentes.

2.2 Primeiro programa Java: Imprimindo uma linha de texto (Cont.)

```
7     public static void main( String args[] )
```

- Ponto de partida de cada aplicativo Java
  - Aplicativos executam a partir do método `main`
    - Parênteses indicam `main` é um método (próximas aulas)
    - Aplicativos Java contém um ou mais métodos
  - Exatamente um método deve ser chamado de `main`
- Métodos podem realizar tarefas e retornar informações
  - `void` significa que `main` não retorna informação
  - Por enquanto, reproduza o `main` em seu código

```
8     {
```

- Chave esquerda começa o corpo da declaração de método
  - Finalizada pela chave direita `}` (linha 11)

Boa prática de programação 2.7

Recue o corpo inteiro de método por um ‘nível’ de recuo entre a chave esquerda, {, e a chave direita, }, o que define o corpo do método. Esse formato faz com que a estrutura do método se destaque tornando a declaração do método mais fácil de ler.

2.2 Primeiro programa Java: Imprimindo uma linha de texto (Cont.)

```
9        System.out.println( "Bem-vindos a programação Java!" );
```

- Instrui o computador a realizar uma ação
  - Imprimir a string de caracteres
    - String – séries de caracteres entre aspas duplas
  - Espaços em branco em strings não são ignorados pelo compilador
- `System.out`
  - Objeto de saída padrão
  - Exibe na janela de comando (Por exemplo, prompt MS-DOS)
- Método `System.out.println`
  - Mostra uma linha de texto
- Essa linha é conhecida como instrução
  - Instruções terminam com ponto-e-vírgula `;`

Erro comum de programação 2.6

Omitir o ponto-e-vírgula no fim de uma instrução é um erro de sintaxe.

Dica de prevenção de erros 2.1

Ao aprender a programar, às vezes é útil ‘quebrar’ um programa funcional para poder familiarizar-se com as mensagens de erro de sintaxe do compilador.

Essas mensagens nem sempre declaram o problema exato no código. Quando você encontrar essas mensagens de erro de sintaxe no futuro, terá uma idéia do que causou o erro.

Tente remover um ponto-e-vírgula ou chave do programa da Figura 2.1 e então recompile o programa para ver as mensagens de erro geradas pela omissão.

Dica de prevenção de erros 2.2

Quando o compilador mostra um erro de sintaxe, o erro pode não ser na linha cujo número é mostrado pela mensagem de erro. Primeiro, verifique a linha na qual o erro foi informado. Se essa linha não contiver erros de sintaxe, verifique as várias linhas anteriores.

2.2 Primeiro programa Java: Imprimindo uma linha de texto (Cont.)

```
11     } // fim do método principal
```

- Fim da declaração de método

```
13 } // fim da classe Welcome1
```

- Fim da declaração de classe
- Pode-se adicionar comentários para indicar método ou classe que a chave termina

Boa prática de programação 2.8

Seguir a chave direita de fechamento (}) do corpo de um método ou de uma declaração de classe com um comentário de fim de linha que indica o método ou declaração de classe que a chave pertence melhora a legibilidade do programa.

2.2 Primeiro programa Java: Imprimindo uma linha de texto (Cont.)

- Compilando o programa
  - Abra uma janela de comando (prompt), e vá ao diretório onde o programa está armazenado
  - Digite `javac Welcome1.java`
  - Se não houver erros de sintaxe, `Welcome1.class` será criado
    - Contém bytecodes que representam o aplicativo
    - Bytecodes serão executados pela JVM

Dica de prevenção de erros 2.3

- Ao tentar compilar um programa, se receber uma mensagem como "`bad command or filename`," "`javac: command not found`" ou "`'javac' is not recognized as an internal or external command, operable program or batch file`," sua instalação de software Java não foi completada adequadamente.

- Se estiver usando o JDK, isso indicará que a variável de ambiente *PATH* do sistema não foi configurada adequadamente.

- Revise as instruções da instalação do JDK em `java.sun.com/j2se/5.0/install.html` cuidadosamente. Em alguns sistemas, depois de corrigir o PATH, talvez seja necessário reinicializar seu computador ou abrir uma nova janela de comando para que essas configurações tenham feito.

Dica de prevenção de erros 2.4

- O compilador Java gera mensagens de erro de sintaxe quando a sintaxe de um programa está incorreta. Cada mensagem de erro contém o nome do arquivo e o número da linha em que o erro ocorreu.
- Por exemplo, `Welcome1.java:6` indica que um erro ocorreu no arquivo `Welcome1.java` na linha 6.
- O restante da mensagem de erro fornece as informações sobre o erro de sintaxe.

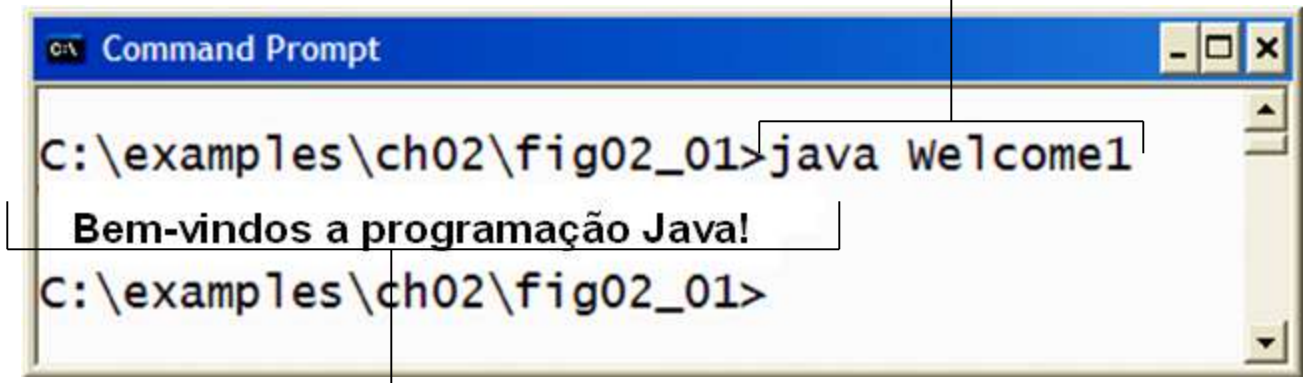
Dica de prevenção de erros 2.5

- A mensagem de erro do compilador "`Public class` NomeDaClasse `must be defined in a file called` NomeDaClasse`.java`" indica que o nome do arquivo não corresponde exatamente ao nome da classe **public** no arquivo ou que o nome da classe foi digitado incorretamente ao compilar a classe.

2.2 Primeiro programa Java: Imprimindo uma linha de texto (Cont.)

- Executando o programa
  - Digite `java Welcome1`
    - Carrega a JVM
    - JVM carrega o arquivo `.class` da classe `Welcome1`
    - `Extensão.class` é omitida do comando
    - JVM chama o método `main`

Você digita esse comando para executar a aplicação



Saída do programa:

Bem-vindos a programação Java!

Dica de prevenção de erros 2.6

- Ao tentar executar um programa Java, se receber uma mensagem como "`Exception in thread "main" java.lang.NoClassDefFoundError: Welcome1`," sua variável de ambiente C*LASSPATH* não foi configurada adequadamente.

- Revise as instruções da instalação do JDK cuidadosamente. Em alguns sistemas, talvez seja necessário reinicializar seu computador ou abrir uma nova janela de comando depois de configurar o *CLASSPATH*.

2.3 Modificando nosso primeiro programa

- Modificar o exemplo da Fig. 2.1 para imprimir o mesmo conteúdo usando código diferente

2.3 Modificando nosso primeiro programa (Cont.)

- Programas modificados
  - `Welcome2.java` (Fig. 2.3) produz a mesma saída que `Welcome1.java` (Fig. 2.1)
  - Usando código diferente

```
9         System.out.print( “Bem-vindos a " );
10        System.out.println( “programação Java!" );
```

  - Linha 9 mostra “Bem-vindos a ” com o cursor na mesma linha
  - Linha 10 mostra “programação Java!” na mesma linha com o cursor sendo apontado para a próxima

Sumário

```
1  // Fig. 2.3: Welcome2.java
2  // Imprimindo uma linha de texto com múltiplas instruções.
3
4  public class Welcome2
5  {
6     // método principal inicia a execução do aplicativo Java
7     public static void main( String args[] )
8     {
9        System.out.print( "Bem-vindos a " );
10       System.out.println( "programação Java!" );
11
12    } // fim do método principal (main)
13
14 } // fim da classe Welcome2
```

```
Bem-vindos a programação Java!
```

`System.out.print` mantém o cursor na mesma linha, então `System.out.println` continua na mesma linha.

Welcome2.java

1. Comentários

2. Linha em branco

3. Início da classe

4.1 Método System.out.println

5. Fim de main, Welcome2

Saída do programa

2.3 Modificando nosso primeiro programa (Cont.)

- Caracteres de escape
  - Barra invertida (\ )
  - Indica que caracteres especiais devem ser impressos
- Caractere de nova linha (\n)
  - Interpretado como "caracteres especiais" pelos métodos `System.out.print` e `System.out.println`
  - Indica que o cursor deve começar na próxima linha
  - `Welcome3.java` (Fig. 2.4)

```
9        System.out.println( "Bem-vindos\na\nprogramação\nJava!" );
```

  - Quebra de linha no \n

Sumário

```
1  // Fig. 2.4: Welcome3.java
2  // Imprimindo múltipas linhas de texto com uma única instrução.
3
4  public class Welcome3
5  {
6     // método principal inicia a execução do aplicativo Java
7     public static void main( String args[] )
8     {
9        System.out.println( "Bem-vindos\na\nprogramação\nJava!" );
10
11    } // fim do método main
12
13 } // fim da classe Welcome3
```

Welcome3.java

1. main
2. System.out.println (usa \n para nova linha)

```
Bem-vindos
a
programação
Java!
```

Saída do programa

Note como uma nova linha é utilizada com cada `\n` .

Seqüência de escape	Descrição
`\n`	Nova linha. Posiciona o cursor de tela no início da próxima linha.
`\t`	Tabulação horizontal. Move o cursor de tela para a próxima parada de tabulação.
`\r`	Retorno de carro. Posiciona o cursor da tela no início da linha atual — não avança para a próxima linha. Qualquer saída de caracteres depois do retorno de carro sobrescreve a saída de caracteres anteriormente gerados na linha atual.
`\\`	Barras invertidas. Utilizadas para imprimir um caractere de barra invertida.
`\"`	Aspas duplas. Utilizadas para imprimir um caractere de aspas duplas. Por exemplo, `System.out.println( "\"entre aspas\"" );` exibe `"entre aspas"`

Fig. 2.5 – Algumas seqüências de escape comuns.

2.4 Exibindo texto com `printf`

- `System.out.printf`
  - Novo recurso a partir do J2SE 5.0
  - Exibe dados formatatos

```
9         System.out.printf( "%s\n%s\n",
10            “Bem-vindos a", “programação Java !" );
```

  - String de formato
    - Texto fixo
    - Especificadores de formato – máscara para um valor
  - Especificador de formato `%s` – máscara para uma string de caracteres

Sumário

```
1  // Fig. 2.6: Welcome4.java
2  // Imprimindo múltiplas linhas em uma caixa de diálogo.
3
4  public class Welcome4
5  {
6     // método principal inicia a execução do aplicativo Java
7     public static void main( String args[] )
8     {
9        System.out.printf( "%s\n%s\n",
10          "Bem-vindos a", "programação Java!" );
11
12    } // fim método main
13
14 } // fim da classe Welcome4
```

`System.out.printf` mostra dados formatados.

- Welcome4.java
- main

```
Bem-vindos a
programação Java!
```

- Saída do programa

Boa prática de programação 2.9

Coloque um espaço depois de cada vírgula (,) em uma lista de argumentos para tornar os programas mais legíveis.

Erro comum de programação 2.7

Dividir uma instrução no meio de um identificador ou de uma string é um erro de sintaxe.

2.5 Outra aplicação Java: Adicionando Inteiros

- Próximo programa
  - Usa classe `Scanner` para ler dois inteiros digitados pelo usuário
  - Usa método `printf` para exibir a soma de dois valores
  - Usa pacotes

```java
// Fig. 2.7: Addition.java
// Programa de adição que exibe a soma de dois números.
import java.util.Scanner; // programa usa a classe Scanne

public class Addition
{
   // método principal inicia a execução do aplicativo Java
   public static void main( String args[] )
   {
      // cria Scanner para obter entrada a partir da janel
      Scanner entrada = new Scanner( System.in );

      int numero1; // primeiro número a somar
      int numero2; // segundo número a somar
      int soma; // soma do numero1 e numero1

      System.out.print( "Digite o primeiro inteiro: " ); // prompt
      numero1 = entrada.nextInt(); // lê o primeiro número do usuário

```

declaração `import` `Scanner` do pacote

Declara e inic `entrada`, a `Scanner`.

Declara variáveis `numero2` e soma

Lê um inteiro do usuário e atribui ao `numero1`.

```
20        System.out.print( "Digite o segundo inteiro: " ); // prompt
21        numero2 = entrada.nextInt(); // lê o segundo número do usuário
22
23        soma = numero1 + numero2; // adiciona os números
24
25        System.out.printf( "Soma é %d\n", soma ); // exibe a soma
26
27     } // fim do método principal (main)
28
29  } // fim da classe Addition
```

Lê um intei
usuário e o a

Calcula son
numero1 e
atribuindo o

Exibe a son
saída forma

Dois inteiros digitados pe
usuário.

```
Digite o primeiro inteiro: 45
Digite o segundo inteiro: 72
Soma é 117
```

2.5 Outra aplicação Java: Adicionando Inteiros (Cont.)

```
3   import java.util.Scanner;  // programa usa a classe Scanner
```

- `Declarações import`
  - Usadas pelo compilador para identificar e localizar classes utilizadas em programas Java
  - Diz ao compilador para carregar a classe `Scanner` do pacote `java.util`

```
5   public class Addition
6   {
```

- Início de `public` class `Addition`
  - Lembre-se que o nome do arquivo deve ser `Addition.java`
- Linhas 8-9: iniciam o método `main`

Erro comum de programação 2.8

Todas as declarações `import` devem aparecer antes da primeira declaração da classe no arquivo. Colocar uma declaração `import` dentro do corpo de uma declaração de classe ou depois de uma declaração de classe é um erro de sintaxe.

Dica de prevençaõ de erros 2.7

Em geral, esquecer-se de incluir uma declaração `import` para uma classe utilizada no seu programa resulta em um erro de compilação contendo uma mensagem como "`cannot resolve symbol.`" Quando isso ocorre, verifique se você forneceu as declarações `import` adequadas e se os nomes nas declarações `import` estão escritos corretamente, incluindo a utilização adequada de letras maiúsculas e minúsculas.

2.5 Outra aplicação Java: Adicionando Inteiros (Cont.)

```
10      // cria Scanner para obter entrada a partir da janela de comando
11      Scanner entrada = new Scanner( System.in );
```

- Declarações de variáveis
- Variáveis
  - Localização na memória que armazena um valor
    - Declare com nome e tipo antes de usá-las
  - `entrada` é um tipo de `Scanner`
    - Permite a um programa ler os dados para utilização
  - Nome da variável: qualquer identificador válido
- Declarações terminam com ponto-e-vírugula `;`
- Inicialize variáveis em sua declaração
  - Sinal de igual (linha 11)
  - Objeto de entrada padrão
    - `System.in`

2.5 Outra aplicação Java: Adicionando Inteiros (Cont.)

```
13        int numero1; // primeiro número a somar
14        int numero2; // segundo número a somar
15         int soma; // soma de numero1 e numero2
```

- Declaração de variáveis `numero1`, `numero2` e `soma` do tipo `int`
  - `int` contém valores inteiros (números integrais): i.e., 0, -4, 97
  - Tipos `float` e `double` contém números reais
  - Tipo `char` contém simples caractere: i.e., x, $, \n, 7
  - `int`, `float`, `double` and `char` are primitive types
- Pode-se adicionar comentários para descrever o propósito das variáveis

```
int numero1, // primeiro número a somar
    numero2, // segundo número a somar
    soma; // soma de numero1 e numero2
```

- Pode-se declarar múltiplas variáveis de mesmo tipo em uma declaração
- Utilize lista separada por vírgulas

Boa prática de programação 2.10

Declare cada variável em uma linha separada. Esse formato permite que um comentário descritivo seja facilmente inserido ao lado de cada declaração.

Boa prática de programação 2.11

Escolher nomes de variáveis significativos ajuda um programa a ser autodocumentado (isto é, pode-se entender o programa simplesmente lendo-o em vez de ler manuais ou visualizar um número excessivo de comentários).

Boa prática de programação 2.12

Por convenção, identificadores de nomes de variáveis iniciam com uma letra minúscula e cada palavra no nome depois da primeira palavra inicia com uma letra maiúscula. Por exemplo, o identificador de nome de variável `firstNumber` tem uma letra `N` maiúscula na sua segunda palavra, `Number`.

2.5 Outra aplicação Java: Adicionando Inteiros (Cont.)

```
17        System.out.print( "Digite o primeiro inteiro: " ); // prompt
```

- Mensagem chamada de prompt – direciona o usuário para uma ação específica
- Pacote `java.lang`

```
18        numero1 = entrada.nextInt(); // lê o primeiro número do usuário
```

- Resultado da chamada do método `nextInt` é colocado na variável `numero1` utilizando o operador de atribuição **=**
  - Instrução de atribuição
  - **=** operador binário – requer dois operandos
    - Expressão na direita avaliada e atribuída a variável da esquerda
  - Lê como: `numero1` pega o valor de `entrada.nextInt()`

Observação de engenharia de software 2.1

Por padrão, o pacote `java.lang` é importado em cada programa Java; portanto, `java.lang` é o único pacote na API do Java que não requer uma declaração `import`.

Boa prática de programação 2.13

Colocar espaços em qualquer um dos lados de um operador binário faz com que eles se destaquem e torna o programa mais legível.

2.5 Outra aplicação Java: Adicionando Inteiros (Cont.)

```
20        System.out.print( "Digite o segundo inteiro: " ); // prompt
```

- Similar a instrução anterior
  - Pede ao usuário para digitar o segundo inteiro

```
21        numero2 = entrada.nextInt(); // lê o segundo número do usuário
```

- Similar a instrução anterior
  - Atribui a variável numero2 o segundo valor digitado

```
23        soma = numero1 + numero2; // adiciona numero1 e numero2
```

- Instrução de atribuição
  - Calcula a soma de `numero1` e `numero2` (lado direito)
  - Utiliza operador de atribuição **=** para atribuir o resultado a variável `soma`
  - Lido como: `soma` pega o valor de `numero1 + numero2`
  - `Numero1` e `numero2` são operandos

2.5 Outra aplicação Java: Adicionando Inteiros (Cont.)

```
25        System.out.printf( "Soma é %d\n: " , soma ); // exibe a soma
```

- Usa `System.out.printf` para exibir o resultado
- Especificador de formato %d
  - Máscara para um valor `int`

```
          System.out.printf( “Soma é %d\n: " , ( numero1 + numero2 ) );
```

- Os cálculos podem ser feitos no próprio `printf`
- Os parênteses na expressão não são necessários

2.6 Conceitos de memória

- Variáveis
  - Cada variável possui um nome, um tipo, tamanho e valor
    - Nome corresponde a localização na memória
  - Quando um novo valor é atribuído a variável, sobrescreve (e destrói) o valor anterior
  - Quando um valor é lido da memória ele não é alterado

Fig. 2.8 – Posição da memória mostrando o nome e o valor da variável `number1`.

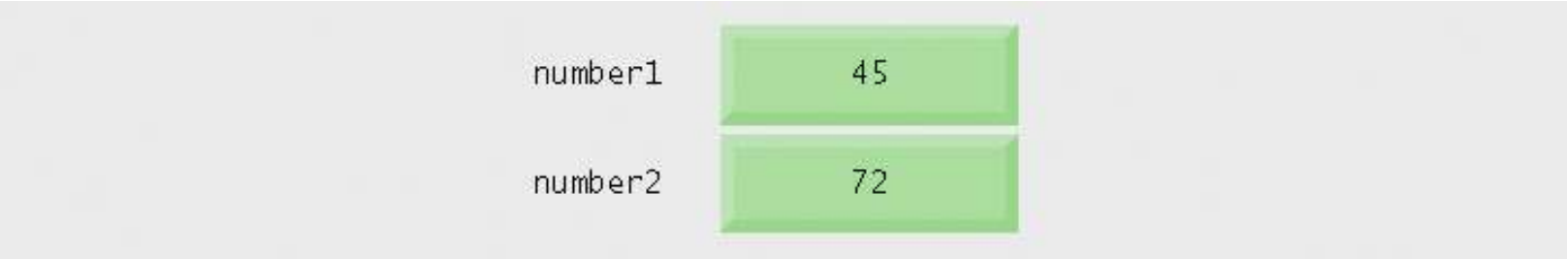


Fig. 2.9 – As posições da memória depois de armazenar os valores para `number1` e `number2`.

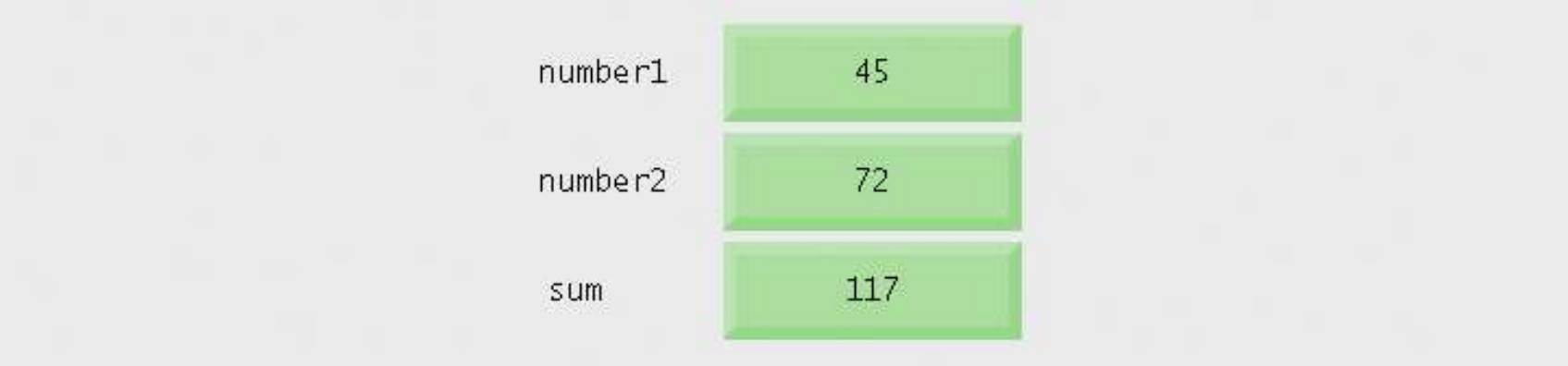


Fig. 2.10 – As posições da memória depois do cálculo e armazenamento.

2.7 Aritmética

- A maioria dos programas realiza cálculos aritméticos
  - Utilização
    - * para multiplicação
    - / para divisão
    - % para resto (ou módulo)
    - +, –
  - Divisão de inteiros trunca o resto

    7 / 5 avaliado como 1
  - Operador módulo % retorna o resto

    7 % 5 avaliado como 2

Operação Java	Operador aritmético	Expressão algébrica	Expressão Java
Adição	`+`	$f + 7$	`f + 7`
Subtração	`-`	$p - c$	`p - c`
Multiplicação	`*`	Bm	`b * m`
Divisão	`/`	x / y ou $\frac{x}{y}$ ou $x \div y$	`x / y`
Resto	`%`	$r \bmod s$	`r % s`

Fig. 2.11 **Operadors aritméticos.**

2.7 Aritmética (Cont.)

- Precedência de operadores
  - Alguns operadoress aritméticos atuam antes de outros (p.ex., multiplicação antes da adição)
    - Use parênteses quando for necessário
  - Exemplo: Encontre a média de três variáveis `a`, `b` e `c`
    - Não use: `a + b + c / 3`
    - Use: `( a + b + c ) / 3`

Operador(es)	Operação(ões)	Ordem de avaliação (precedência)
*	Multiplicação	Avaliado primeiro.Se houver vários operadores desse tipo, eles são avaliados da esquerda para a direita.
/	Divisão	
%	Resto	
+	Adição	Avaliado em seguida. Se houver vários operadores desse tipo, eles são avaliados da esquerda para a direita.
-	Subtração	

Fig. 2.12 **Precedência de operadores aritméticos.**

Boa prática de programação 2.14

Utilizar parênteses para expressões aritméticas complexas, mesmo quando os parênteses não são necessários, pode tornar as expressões aritméticas mais fáceis de ler.

Passo 1.	y = 2 * 5 * 5 + 3 * 5 + 7; 2 * 5 = 10	(Multiplicação mais à esquerda)
Passo 2.	y = 10 * 5 + 3 * 5 + 7; 10 * 5 = 50	(Multiplicação mais à esquerda)
Passo 3.	y = 50 + 3 * 5 + 7; 3 * 5 = 15	(Multiplicação antes da adição)
Passo 4.	y = 50 + 15 + 7; 50 + 15 = 65	(Adição mais à esquerda)
Passo 5.	y = 65 + 7; 65 + 7 = 72	(Última adição)
Passo 6.	y = 72	(Última operação — coloca 72 em y)

Fig. 2.13 **Ordem em que um polinômio de segundo grau é avaliado.**

2.8 Tomada de decisão: operadores de igualdade e operadores relacionais

- Condição
  - Expressão que pode ser `true` ou `false`
- `Instrução if`
  - Versão simples nessa seção, mais detalhes depois
  - Se uma condição é `true`, então o corpo da instrução `if` é executado
  - As condições em instruções `if` podem ser formadas utilizando operadores de igualdade ou operadores relacionais (próximo *slide*)

Padrão algébrico operador de igualdade ou relacional	Operador de igualdade ou relacional em Java ou operador relacional	Exemplo de condição em Java	Significado da condição em Java
Operadores de igualdade			
=	==	x == y	x é igual y
≠	!=	x != y	x não é igual a y
Operadores relacionais			
>	>	x > y	x é maior que y
<	<	x < y	x é menor que y
≥	>=	x >= y	x é maior que ou igual a y
≤	<=	x <= y	x é menor que ou igual a y

Fig. 2.14 | **Operadores de igualdade e operadores relacionais.**

```java
// Fig. 2.15: Comparison.java
// Compara inteiros utilizando if, operadores relaicionais
// e operadores de igualdade.
import java.util.Scanner; // programa utiliza a classe Scanner

public class Comparison
{
   // método principal inicia a execução do programa
   public static void main( String args[] )
   {
      // cria Scanner para obter entrada a partir da janela de comando
      Scanner lerDados = new Scanner( System.in );

      int number1; // primeiro número a comparar
      int number2; // segundo número a comparar

      System.out.print( "Digite primeiro inteiro: " ); // prompt
      number1 = lerDados.nextInt(); // lê primeiro número do usuário

      System.out.print( "Digite segundo inteiro: " ); // prompt
      number2 = lerDados.nextInt(); // lê segundo número do usuário

      if ( number1 == number2 )
         System.out.printf( "%d == %d\n", numb

      if ( number1 != number2 )
         System.out.printf( "%d != %d\n", number1, number2 );

      if ( number1 < number2 )
         System.out.printf( "%d < %d\n", numb
```

Teste de igualdade, exibe o resultado usando `printf`.

Compara dois números usando operador relacional <.

```
31
32      if ( number1 > number2 )
33         System.out.printf( "%d > %d\n", number1, number2 );
34
35      if ( number1 <= number2 )
36         System.out.printf( "%d <= %d\n", number1,
37
38      if ( number1 >= number2 )
39         System.out.printf( "%d >= %d\n", number1, number2 );
40
41   } // fim do método main
42
43 } // fim da classe Comparison
```

Compara dois números usando operador relacional >, <= e >=.

```
Digite o primeiro inteiro: 777
Digite o segundo inteiro: 777
777 == 777
777 <= 777
777 >= 777
```

```
Digite o primeiro inteiro: 1000
Digite o segundo inteiro: 2000
1000 != 2000
1000 < 2000
1000 <= 2000
```

```
Digite o primeiro inteiro: 2000
Digite o segundo inteiro: 1000
2000 != 1000
2000 > 1000
2000 >= 1000
```

2.8 Tomada de decisão: operadores de igualdade e operadores relacionais (Cont.)

- Linha 6: declaração da classe `Comparison` inicia nessa linha
- Linha 12: declara a variável Scanner lerDados e lhe atribui um Scanner que insere dados a partir da entrada-padrão
- Linhas 14-15: declaram variáveis tipo `int`
- Linhas 17-18: solicitam que o usuário digite o primeiro inteiro e insira o valor
- Linhas 20-21: solicitam que o usuário digite o segundo inteiro e insira o valor

2.8 Tomada de decisão: operadores de igualdade e operadores relacionais

```
23        if ( number1 == number2 )
24           System.out.printf( "%d == %d\n", number1, number2 );
```

- `Instrução if` que compara um valor usando ==
  - Se variáveis forem iguais (condição true)
    - Linha 24 executa
  - Se variáveis não forem iguais, instrução é pulada
  - Não há ponto-e-vírugla no fim de uma instrução `if`
  - Instrução vazia
    - Nenhuma tarefa é executada
- Linhas 26-27, 29-30, 32-33, 35-36 e 38-39
  - Comparam `number1` e `number2` com os operadores `!=`, <, >, <= e >=, respectivamente

Erro comum de programação 2.9

Esquecer o parêntese esquerdo e/ou direito para a condição em uma estrutura `if` é um erro de sintaxe – os parênteses são requeridos.

Erro comum de programação 2.10

Confundir o operador de igualdade, ==, com o operador de atribuição, =, pode causar um erro de lógica ou um erro de sintaxe. O operador de igualdade deve ser lido como ‘igual a’, e o operador de atribuição deve ser lido como ‘obtém’ ou ‘obtém o valor de’. Para evitar confusão, algumas pessoas lêem o operador de igualdade como ‘duplo igual’ ou ‘igual igual’.

Erro comum de programação 2.11

É um erro de sintaxe se os operadores ==, !=, >= e <= contiverem espaços entre seus símbolos como em = =, ! =, > = e < =, respectivamente.

Erro comum de programação 2.12

Inverter os operadores !=, >= e <= como em =!, => e =<, é um erro de sintaxe.

Boa prática de programação 2.15

Recue um corpo da estrutura `if` para fazer com que ele se destaque e para aprimorar a legibilidade do programa.

Boa prática de programação 2.16

Coloque apenas uma instrução por linha em um programa. Esse formato aprimora a legibilidade do programa.

Erro comum de programação 2.13

Colocar um ponto-e-vírgula imediatamente depois do parêntese direito da condição em uma instrução `if` é normalmente um erro de lógica.

Boa prática de programação 2.17

Uma instrução prolongada pode ser estendida por várias linhas. Se uma única instrução deve ser dividida em várias linhas, escolha dividi-la em pontos que façam sentido, como depois de uma vírgula em uma lista separada por vírgulas ou depois de um operador em uma expressão longa. Se uma instrução for dividida em duas ou mais linhas, recue todas as linhas subseqüentes até o fim da instrução.

Boa prática de programação 2.18

Consulte a tabela de precedência de operadores ao escrever expressões que contém vários operadores. Confirme se as operações na expressão são realizadas na ordem em que você espera. Se você não tiver certeza da ordem de avaliação em uma expressão complexa, utilize parênteses para forçar a ordem, exatamente como faria em expressões algébricas. Observe que alguns operadores, como o de atribuição, =, associam da direita para a esquerda e não da esquerda para a direita.

Operadores				Associatividade	Tipo
*	/	%		da esquerda para a direita	multiplicativo
+	-			da esquerda para a direita	aditivo
<	<=	>	>=	da esquerda para a direita	relacional
==	!=			da esquerda para a direita	igualdade
=				da direita para a esquerda	atribuição

Fig. 2.16 | **Operações de precedência e de associatividade discutidas.**

Referências

- Adaptado de: “Teaching Resources for Java How to Program, 6/e”.
- Deitel & Deitel. Java como programar, 6 ed.